

Classification Assignment

Problem Statement or Requirement:

A requirement from the Hospital, Management asked us to create a predictive model which will predict the chronic kidney disease (CKD) based on the several parameters. The Client has provided the dataset of the same.

1.) Identify your problem statement

Here We need to find the chronic kidney disease is present or not. So the output is categorical data.

Machine Learning-> Supervised learning -> Classification

2) Tell basic info about the dataset (Total number of rows, columns)

Columns = 25

Rows= 399

Independent variable = 24 variables

Dependent variable = 1

3) Mention the pre-processing method if you're doing any (like converting string to number – nominal data)

Ordinal Method – used to convert for categorical data rbc, pc...etc.,

4) Develop a good model with good evaluation metric. You can use any machine learning algorithm; you can create many models. Finally, you must come up with final model.

1) Decision Tree:

Accuracy: 0.99

```
7]: from sklearn.metrics import confusion_matrix
cm = confusion_matrix(Y_test, Y_pred)
cm
```

```
7]: array([[45,  0],
        [ 1, 74]])
```

```
8]: from sklearn.metrics import classification_report
clf_report = classification_report(Y_test, Y_pred)
print(clf_report)
```

	precision	recall	f1-score	support
0	0.98	1.00	0.99	45
1	1.00	0.99	0.99	75
accuracy			0.99	120
macro avg	0.99	0.99	0.99	120
weighted avg	0.99	0.99	0.99	120

```
[4]: grid.best_estimator_
```

[4]:

DecisionTreeClassifier		
▼ Parameters		
	<code>criterion</code>	<code>'entropy'</code>
	<code>splitter</code>	<code>'random'</code>
	<code>max_depth</code>	<code>None</code>
	<code>min_samples_split</code>	<code>10</code>
	<code>min_samples_leaf</code>	<code>1</code>
	<code>min_weight_fraction_leaf</code>	<code>0.0</code>
	<code>max_features</code>	<code>'log2'</code>
	<code>random_state</code>	<code>0</code>
	<code>max_leaf_nodes</code>	<code>None</code>
	<code>min_impurity_decrease</code>	<code>0.0</code>
	<code>class_weight</code>	<code>None</code>
	<code>ccp_alpha</code>	<code>0.0</code>
	<code>monotonic_cst</code>	<code>None</code>

2. Random Forest

Accuracy = 0.98

```
[4]: from sklearn.metrics import confusion_matrix
cm = confusion_matrix(Y_test, Y_pred)
cm
```

```
[4]: array([[44,  1],
        [ 1, 74]])
```

```
[5]: from sklearn.metrics import classification_report
clf_report = classification_report(Y_test, Y_pred)
print(clf_report)
```

	precision	recall	f1-score	support
0	0.98	0.98	0.98	45
1	0.99	0.99	0.99	75
accuracy			0.98	120
macro avg	0.98	0.98	0.98	120
weighted avg	0.98	0.98	0.98	120

```
[ ]:
```

3. SVM

Accuracy = 0.98

Fitting 2 folds for each of 1 candidates, totalling 2 fits

```
[3]: from sklearn.metrics import confusion_matrix
cm= confusion_matrix(Y_test, Y_pred)
cm
```

```
[3]: array([[44,  1],
        [ 1, 74]])
```

```
[4]: grid.best_estimator_
```

```
[4]: SVC
Parameters
```

```
[5]: from sklearn.metrics import classification_report
clr= classification_report(Y_test, Y_pred)
print(clr)
```

	precision	recall	f1-score	support
0	0.98	0.98	0.98	45
1	0.99	0.99	0.99	75
accuracy			0.98	120
macro avg	0.98	0.98	0.98	120
weighted avg	0.98	0.98	0.98	120

```
[ ]:
```

```
[4]: grid.best_estimator_
```

[4]:

SVC		
▼ Parameters		
	C	1
	kernel	'linear'
	degree	3
	gamma	'scale'
	coef0	0.0
	shrinking	True
	probability	True
	tol	0.001
	cache_size	200
	class_weight	None
	verbose	False
	max_iter	-1
	decision_function_shape	'ovr'
	break_ties	False
	random_state	0

```
[5]: from sklearn.metrics import classification_report
```

```
[6]: grid.best_estimator_
```

[6]:

RandomForestClassifier		
Parameters		
	n_estimators	50
	criterion	'gini'
	max_depth	5
	min_samples_split	5
	min_samples_leaf	1
	min_weight_fraction_leaf	0.0
	max_features	'log2'
	max_leaf_nodes	None
	min_impurity_decrease	0.0
	bootstrap	True
	oob_score	False
	n_jobs	-1
	random_state	0
	verbose	0
	warm_start	False
	class_weight	None
	ccp_alpha	0.0
	max_samples	None
	monotonic_cst	None

4) Logistic Regression:

Accuracy = 0.97

```
[3]: Y_pre= grid.predict(X_test)
```

```
from sklearn.metrics import confusion_matrix  
cm = confusion_matrix(Y_test, Y_pre)
```

```
[4]: print(cm)  
from sklearn.metrics import classification_report  
print(classification_report(Y_test, Y_pre))
```

```
[[44  1]  
 [ 2 73]]
```

	precision	recall	f1-score	support
0	0.96	0.98	0.97	45
1	0.99	0.97	0.98	75
accuracy			0.97	120
macro avg	0.97	0.98	0.97	120
weighted avg	0.98	0.97	0.98	120

```
[ ]:
```

5) KNN

Accuracy = 0.80


```
[3]: Y_pred= grid.predict(X_test)

from sklearn.metrics import confusion_matrix
cmx= confusion_matrix(Y_test, Y_pred)
print(cmx)
```

```
[[33  3]
 [17 47]]
```

```
[4]: from sklearn.metrics import classification_report
cr = classification_report(Y_test, Y_pred)
print(cr)
```

	precision	recall	f1-score	support
0	0.66	0.92	0.77	36
1	0.94	0.73	0.82	64
accuracy			0.80	100
macro avg	0.80	0.83	0.80	100
weighted avg	0.84	0.80	0.80	100

```
5]:
```

5]:

KNeighborsClassifier		
Parameters		
n_neighbors		7
weights		'distance'
algorithm		'auto'
leaf_size		20
p		1
metric		'manhattan'
metric_params		None
n_jobs		None

6. Naïve Bayes:

Accuracy =0.97

```

grid=grid.fit(X_train, Y_train)

3]: Y_pred= grid.predict(X_test)
    from sklearn.metrics import confusion_matrix
    cm= confusion_matrix(Y_test, Y_pred)
    print(cm)

[[45  0]
 [ 3 72]]

4]: from sklearn.metrics import classification_report
    clr= classification_report(Y_test,Y_pred)
    print(clr)

```

	precision	recall	f1-score	support
0	0.94	1.00	0.97	45
1	1.00	0.96	0.98	75
accuracy			0.97	120
macro avg	0.97	0.98	0.97	120
weighted avg	0.98	0.97	0.98	120

```

]:

```

```

[5]: grid.best_estimator_

```

[5]:

GaussianNB

▼ Parameters

priors	None
var_smoothing	np.float64(1e-12)

```

[ ]:

```

7. XGBoost

Accuracy = 0.98

```
grid= grid.fit(X_train,Y_train)
Y_pred = grid.predict(X_test)
```

Fitting 5 folds for each of 36 candidates, totalling 180 fits

```
[3]: from sklearn.metrics import confusion_matrix
cm= confusion_matrix(Y_test, Y_pred)
print(cm)
```

```
[[44  1]
 [ 1 74]]
```

```
[4]: from sklearn.metrics import classification_report
clr= classification_report(Y_test,Y_pred)
print(clr)
```

	precision	recall	f1-score	support
0	0.98	0.98	0.98	45
1	0.99	0.99	0.99	75
accuracy			0.98	120
macro avg	0.98	0.98	0.98	120
weighted avg	0.98	0.98	0.98	120

.

Overall, **Decision Tree is best model:**

Accuracy: 0.99